Высокопроизвольдитиельные реализации метода сопряжённых градиентов для решения некорректно поставленнных линейных обратных задач

Ли Вэйчжи

liveychzhi@my.msu.ru

Кафедра математики, Физический факультет,
МГУ имени М.В. Ломоносова

13 Мая, 2026
Москва, Россия

1. Постановка задачи

Многолинейные обратные задачи сводятся к необходимости решения систем линейных алгебры.

$$A\,x = b$$

Для решения задач такого типа существует огромное количество различных алгоритмов.Одни из самых надёжных алгоритмов - это метод Сопряжённых Градиентов.

1. Постановка задачи

Многолинейные обратные задачи сводятся к необходимости решения систем линейных алгебры.

$$A\,x = b$$

Для решения задач такого типа существует огромное количество различных алгоритмов.Одни из самых надёжных алгоритмов - это метод Сопряжённых Градиентов.

1. Постановка задачи

Многолинейные обратные задачи сводятся к необходимости решения систем линейных алгебры.

$$A\,x = b$$

Для решения задач такого типа существует огромное количество различных алгоритмов.Одни из самых надёжных алгоритмов - это метод Сопряжённых Градиентов.

1. Постановка задачи

Многолинейные обратные задачи сводятся к необходимости решения систем линейных алгебры.

$$A\,x = b$$

Для решения задач такого типа существует огромное количество различных алгоритмов.Одни из самых надёжных алгоритмов - это метод Сопряжённых Градиентов.

2. Методы Сопряжённых Градиентов

Существует множество реализаций метода сопряжённых градиентов. Рассмотрим два популярных алгоритма:

Алгоритм 1

Require: $A, b, x^{(1)}, \alpha$
Ensure: x
  $s \leftarrow 1;\ p \leftarrow 0$
  while *True* **do**
    if $s = 1$ **then**
      $r \leftarrow A^{\mathsf{T}}(Ax - b) + \alpha x$
    else
      $r \leftarrow r - q/(p, q)$
    end if
    $p \leftarrow p + r/(r, r)$
    $q \leftarrow A^{\mathsf{T}}(Ap) + \alpha p$
    $x \leftarrow x - p/(p, q)$
    $s \leftarrow s + 1$
  end while

Алгоритм 2

Require: $A, b, x^{(1)}, \alpha$
Ensure: x
  $s \leftarrow 1;\ r \leftarrow b - Ax$
  $z \leftarrow A^{\mathsf{T}} r;\ p \leftarrow z$
  while *True* **do**
    $k_1 \leftarrow \frac{(z,z)}{(Ap,Ap)+\alpha(p,p)}$
    $x \leftarrow x + k_1 p$
    $r \leftarrow r - k_1 Ap$
    $k_2 \leftarrow (z, z)$
    $z \leftarrow A^{\mathsf{T}} r - \alpha x$
    $k_2 \leftarrow (z, z)/k_2$
    $p \leftarrow z + k_2 p$
    $s \leftarrow s + 1$
  end while

Какой алгоритм лучше на практике?

3.Почему возник такой вопрос?(Какой алгоритм лучше на практике?)

С точки зрения теории эти алгоритмы одинаковые.
С точки зрения практики эти алгоритмы разные.

- 1 проблема: Формулы разные , это значит ошибка машинного округление существует.
- 2 проблема:одинаково ли хорошо эти алгоритмы распараллеливаются ?

результат будет разный

3.Почему возник такой вопрос?(Какой алгоритм лучше на практике?)

С точки зрения теории эти алгоритмы одинаковые.
С точки зрения практики эти алгоритмы разные.

- 1 проблема: Формулы разные , это значит ошибка машинного округление существует.
- 2 проблема:одинаково ли хорошо эти алгоритмы распараллеливаются ?

результат будет разный

3.Почему возник такой вопрос?(Какой алгоритм лучше на практике?)

С точки зрения теории эти алгоритмы одинаковые.
С точки зрения практики эти алгоритмы разные.

- 1 проблема: Формулы разные , это значит ошибка машинного округление существует.
- 2 проблема:одинаково ли хорошо эти алгоритмы распараллеливаются ?

результат будет разный

4. Задача моей работы

T_1 — время работы последовательной версии программы на одном вычислительном узле (процессоре, ядре)

T_n — время работы программы на системе из n вычислительных узлов

$S_n = \frac{T_1}{T_n}$ — ускорение работы программы

$E_n = \frac{S_n}{n}$ — эффективность распараллеливания

Моя задача - определим какой алгоритм имеет более эффективную программную реализацию.

4. Задача моей работы

T_1 — время работы последовательной версии программы на одном вычислительном узле (процессоре, ядре)

T_n — время работы программы на системе из n вычислительных узлов

$S_n = \frac{T_1}{T_n}$ — ускорение работы программы

$E_n = \frac{S_n}{n}$ — эффективность распараллеливания

Моя задача - определим какой алгоритм имеет более эффективную программную реализацию.

4. Задача моей работы

T_1 — время работы последовательной версии программы на одном вычислительном узле (процессоре, ядре)

T_n — время работы программы на системе из n вычислительных узлов

$S_n = \frac{T_1}{T_n}$ — ускорение работы программы

$E_n = \frac{S_n}{n}$ — эффективность распараллеливания

Моя задача - определим какой алгоритм имеет более эффективную программную реализацию.

4. Задача моей работы

T_1 — время работы последовательной версии программы на одном вычислительном узле (процессоре, ядре)

T_n — время работы программы на системе из n вычислительных узлов

$S_n = \frac{T_1}{T_n}$ — ускорение работы программы

$E_n = \frac{S_n}{n}$ — эффективность распараллеливания

Моя задача - определим какой алгоритм имеет более эффективную программную реализацию.

Эффективность и масштабируемость

Эффективность

Способность оценить, насколько полно используются вычислительные ресурсы.

$E \approx 1$ — ресурсы используются полностью
$E \approx 0$ — ресурсы практически не используются

Масштабируемость

Способность программы сохранять эффективность при увеличении числа вычислительных узлов.

Различают **сильную** и **слабую** масштабируемость

Цель работы

Разработать универсальную методологию, позволяющую объективно сравнить параллельные алгоритмы по:

- Практической эффективности распараллеливания
- Показателям сильной масштабируемости

Алгоритм 1

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,A}$

Инициализация: $C_1 M$

Шаг $s = 1$:

$$T_{s=1} = C_1(6MN+4M+8N)+\tilde{C}_1(4MN)$$

Шаг $s > 1$:

$$T_{stable} = C_1(4MN+3M+12N)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,A}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы (q): $C_2 N_{loc} \log_2 n_1$
- Скаляры (1 условие + 3 произведения): $4C_2 \log_2 n$

Составляющие $T_{n,A}$

$$T_{n,A} = T_{pre} + T_{s=1} + (s-1)T_{stable}$$

$$E_{n,A} = \frac{T_{1,A}}{n \cdot T_{n,A}}$$

Алгоритм 1

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,A}$

Инициализация: $C_1 M$

Шаг $s = 1$:

$$T_{s=1} = C_1(6MN+4M+8N)+\tilde{C}_1(4MN)$$

Шаг $s > 1$:

$$T_{stable} = C_1(4MN+3M+12N)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,A}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы (q): $C_2 N_{loc} \log_2 n_1$
- Скаляры (1 условие + 3 произведения): $4C_2 \log_2 n$

Составляющие $T_{n,A}$

$$T_{n,A} = T_{pre} + T_{s=1} + (s-1)T_{stable}$$

$$E_{n,A} = \frac{T_{1,A}}{n \cdot T_{n,A}}$$

Алгоритм 1

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,A}$

Инициализация: $C_1 M$

Шаг $s = 1$:

$$T_{s=1} = C_1(6MN+4M+8N)+\tilde{C}_1(4MN)$$

Шаг $s > 1$:

$$T_{stable} = C_1(4MN+3M+12N)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,A}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы (q): $C_2 N_{loc} \log_2 n_1$
- Скаляры (1 условие + 3 произведения): $4C_2 \log_2 n$

Составляющие $T_{n,A}$

$$T_{n,A} = T_{pre} + T_{s=1} + (s-1)T_{stable}$$

$$E_{n,A} = \frac{T_{1,A}}{n \cdot T_{n,A}}$$

Алгоритм 1

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,A}$

Инициализация: $C_1 M$

Шаг $s = 1$:

$$T_{s=1} = C_1(6MN+4M+8N)+\tilde{C}_1(4MN)$$

Шаг $s > 1$:

$$T_{stable} = C_1(4MN+3M+12N)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,A}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы (q): $C_2 N_{loc} \log_2 n_1$
- Скаляры (1 условие + 3 произведения): $4C_2 \log_2 n$

Составляющие $T_{n,A}$

$$T_{n,A} = T_{pre} + T_{s=1} + (s-1)T_{stable}$$

$$E_{n,A} = \frac{T_{1,A}}{n \cdot T_{n,A}}$$

Алгоритм 1

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,A}$

Инициализация: $C_1 M$

Шаг $s = 1$:

$$T_{s=1} = C_1(6MN+4M+8N)+\tilde{C}_1(4MN)$$

Шаг $s > 1$:

$$T_{stable} = C_1(4MN+3M+12N)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,A}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы (q): $C_2 N_{loc} \log_2 n_1$
- Скаляры (1 условие + 3 произведения): $4C_2 \log_2 n$

Составляющие $T_{n,A}$

$$T_{n,A} = T_{pre} + T_{s=1} + (s-1)T_{stable}$$

$$\boxed{E_{n,A} = \frac{T_{1,A}}{n \cdot T_{n,A}}}$$

Алгоритм 2

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,B}$

Инициализация:

$$T_{init} = C_1(2MN+2M+2N)+\tilde{C}_1(2MN)$$

Итерация:

$$T_{iter} = C_1(4MN+5M+8N+2)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,B}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы $(A^T r)$: $C_2 N_{loc} \log_2 n_1$
- Скаляры: только $C_2 \log_2 n_2$ (1 раз!)

$$E_{n,B} = \frac{T_{1,B}}{n \cdot T_{n,B}}$$

Алгоритм 2

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,B}$

Инициализация:

$$T_{init} = C_1(2MN+2M+2N)+\tilde{C}_1(2MN)$$

Итерация:

$$T_{iter} = C_1(4MN+5M+8N+2)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,B}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы $(A^T r)$: $C_2 N_{loc} \log_2 n_1$
- Скаляры: только $C_2 \log_2 n_2$ (1 раз!)

$$E_{n,B} = \frac{T_{1,B}}{n \cdot T_{n,B}}$$

Алгоритм 2

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,B}$

Инициализация:

$$T_{init} = C_1(2MN+2M+2N)+\tilde{C}_1(2MN)$$

Итерация:

$$T_{iter} = C_1(4MN+5M+8N+2)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,B}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы $(A^T r)$: $C_2 N_{loc} \log_2 n_1$
- Скаляры: только $C_2 \log_2 n_2$ (1 раз!)

$$E_{n,B} = \frac{T_{1,B}}{n \cdot T_{n,B}}$$

Алгоритм 2

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,B}$

Инициализация:

$$T_{init} = C_1(2MN+2M+2N)+\tilde{C}_1(2MN)$$

Итерация:

$$T_{iter} = C_1(4MN+5M+8N+2)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,B}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы $(A^T r)$: $C_2 N_{loc} \log_2 n_1$
- Скаляры: только $C_2 \log_2 n_2$ (1 раз!)

$$E_{n,B} = \frac{T_{1,B}}{n \cdot T_{n,B}}$$

Алгоритм 2

Модель стоимости

- C_1 — время FLOP-операции
- $\tilde{C}_1$ — штраф непоследовательного доступа
- $T_{comm} = C_2 \cdot V \cdot \log_2 n$

Последовательное время $T_{1,B}$

Инициализация:

$$T_{init} = C_1(2MN+2M+2N)+\tilde{C}_1(2MN)$$

Итерация:

$$T_{iter} = C_1(4MN+5M+8N+2)+\tilde{C}_1(2MN)$$

Параллельное время $T_{n,B}$

Вычисления: замена $MN \to \frac{MN}{n}$

Коммуникации:

- Строки (Ax, Ap): $2C_2 M_{loc} \log_2 n_2$
- Столбцы $(A^T r)$: $C_2 N_{loc} \log_2 n_1$
- Скаляры: только $C_2 \log_2 n_2$ (1 раз!)

$$E_{n,B} = \frac{T_{1,B}}{n \cdot T_{n,B}}$$

Итоговые формулы эффективности

Алгоритм A: полная формула $E_{n,A}$

$$E_{n,A} = \frac{C_1 M + \left[C_1(6MN + 4M + 8N) + \tilde{C}_1(4MN)\right] + (s-1)\left[C_1(4MN + 3M + 12N) + \tilde{C}_1(2MN)\right]}{n \cdot \left[C_1 M_{loc} + \Delta T_{n,A}^{s=1} + (s-1)\Delta T_{n,A}^{stable}\right]}$$

где:

$$\Delta T_{n,A}^{s=1} = C_1\left(6\frac{MN}{n} + 4M_{loc} + 8N_{loc}\right) + \tilde{C}_1\left(4\frac{MN}{n} - 2N_{loc}\right) + C_2\left(3M_{loc}\ln_2 + 2N_{loc}\ln_1 + \ln_1 + 2\ln_2\right)$$

$$\Delta T_{n,A}^{stable} = C_1\left(4\frac{MN}{n} + 3M_{loc} + 12N_{loc}\right) + \tilde{C}_1\left(2\frac{MN}{n} - N_{loc}\right) + C_2\left(2M_{loc}\ln_2 + N_{loc}\ln_1 + \ln_1 + 3\ln_2\right)$$

Алгоритм B: полная формула $E_{n,B}$

$$E_{n,B} = \frac{\left[C_1(2MN + 2M + 2N) + \tilde{C}_1(2MN)\right] + s\left[C_1(4MN + 5M + 8N + 2) + \tilde{C}_1(2MN)\right]}{n \cdot \left[\Delta T_{n,B}^{init} + s \cdot \Delta T_{n,B}^{iter}\right]}$$

где:

$$\Delta T_{n,B}^{init} = C_1\left(2\frac{MN}{n} + M_{loc} + N_{loc}\right) + \tilde{C}_1\left(\frac{2MN}{n} - N_{loc}\right) + C_2\left(M_{loc}\ln_2 + N_{loc}\ln_1\right)$$

$$\Delta T_{n,B}^{iter} = C_1\left(4\frac{MN}{n} + 7M_{loc} + 8N_{loc} + 2\right) + \tilde{C}_1\left(2\frac{MN}{n} - N_{loc}\right) + C_2\left(2M_{loc}\ln_2 + N_{loc}\ln_1 + 2\ln_1 + \ln_2\right)$$

Итоговые формулы эффективности

Алгоритм A: полная формула $E_{n,A}$

$$E_{n,A} = \frac{C_1 M + \left[C_1(6MN + 4M + 8N) + \tilde{C}_1(4MN)\right] + (s-1)\left[C_1(4MN + 3M + 12N) + \tilde{C}_1(2MN)\right]}{n \cdot \left[C_1 M_{loc} + \Delta T_{n,A}^{s=1} + (s-1)\Delta T_{n,A}^{stable}\right]}$$

где:

$$\Delta T_{n,A}^{s=1} = C_1\left(6\frac{MN}{n} + 4M_{loc} + 8N_{loc}\right) + \tilde{C}_1\left(4\frac{MN}{n} - 2N_{loc}\right) + C_2\left(3M_{loc}\ln_2 + 2N_{loc}\ln_1 + \ln_1 + 2\ln_2\right)$$

$$\Delta T_{n,A}^{stable} = C_1\left(4\frac{MN}{n} + 3M_{loc} + 12N_{loc}\right) + \tilde{C}_1\left(2\frac{MN}{n} - N_{loc}\right) + C_2\left(2M_{loc}\ln_2 + N_{loc}\ln_1 + \ln_1 + 3\ln_2\right)$$

Алгоритм B: полная формула $E_{n,B}$

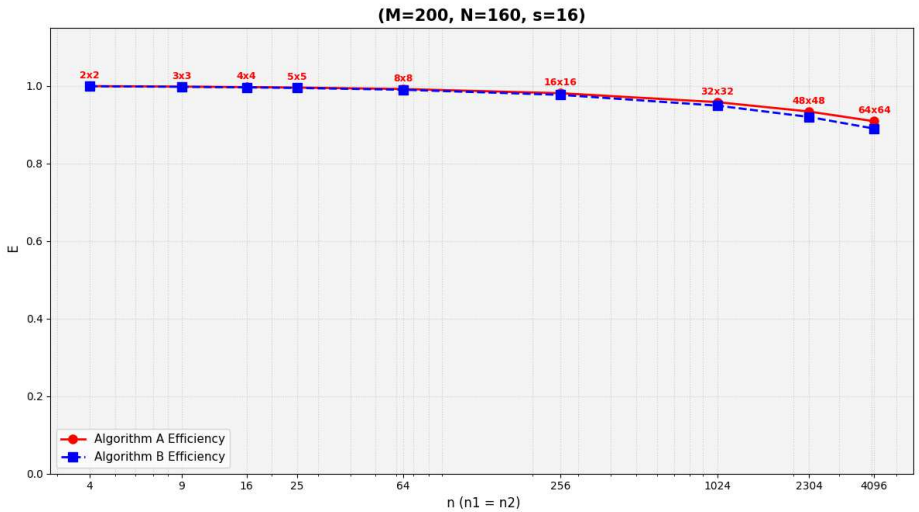
$$E_{n,B} = \frac{\left[C_1(2MN + 2M + 2N) + \tilde{C}_1(2MN)\right] + s\left[C_1(4MN + 5M + 8N + 2) + \tilde{C}_1(2MN)\right]}{n \cdot \left[\Delta T_{n,B}^{init} + s \cdot \Delta T_{n,B}^{iter}\right]}$$

где:

$$\Delta T_{n,B}^{init} = C_1\left(2\frac{MN}{n} + M_{loc} + N_{loc}\right) + \tilde{C}_1\left(\frac{2MN}{n} - N_{loc}\right) + C_2\left(M_{loc}\ln_2 + N_{loc}\ln_1\right)$$

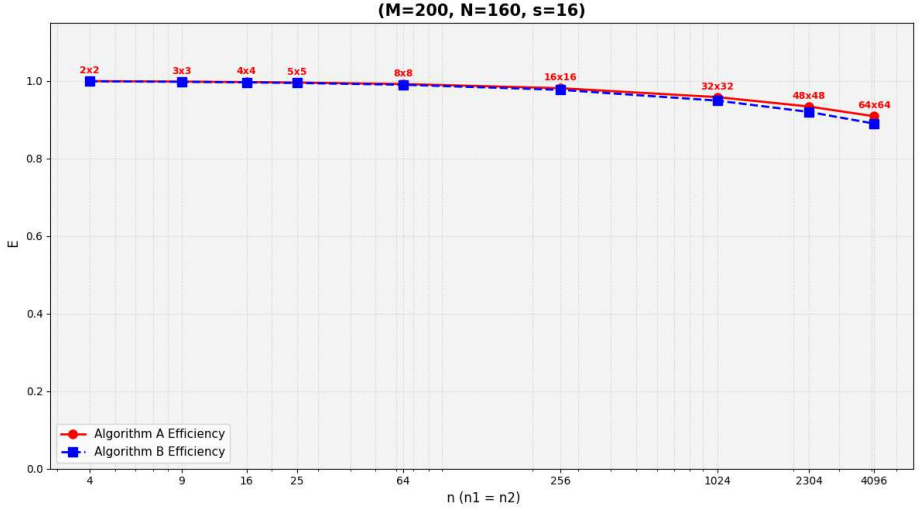
$$\Delta T_{n,B}^{iter} = C_1\left(4\frac{MN}{n} + 7M_{loc} + 8N_{loc} + 2\right) + \tilde{C}_1\left(2\frac{MN}{n} - N_{loc}\right) + C_2\left(2M_{loc}\ln_2 + N_{loc}\ln_1 + 2\ln_1 + \ln_2\right)$$

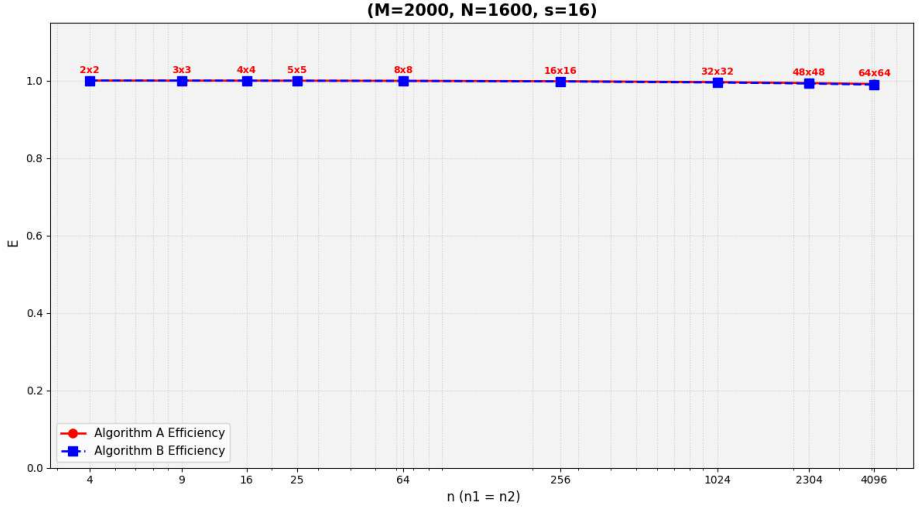
Результаты эксперимента: сравнение эффективности



$M = 200, \ N = 160$

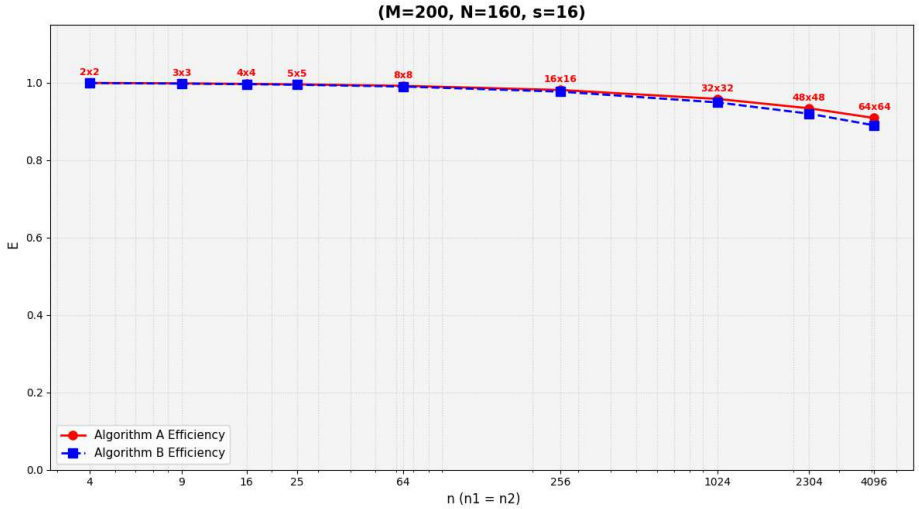
Результаты эксперимента: сравнение эффективности

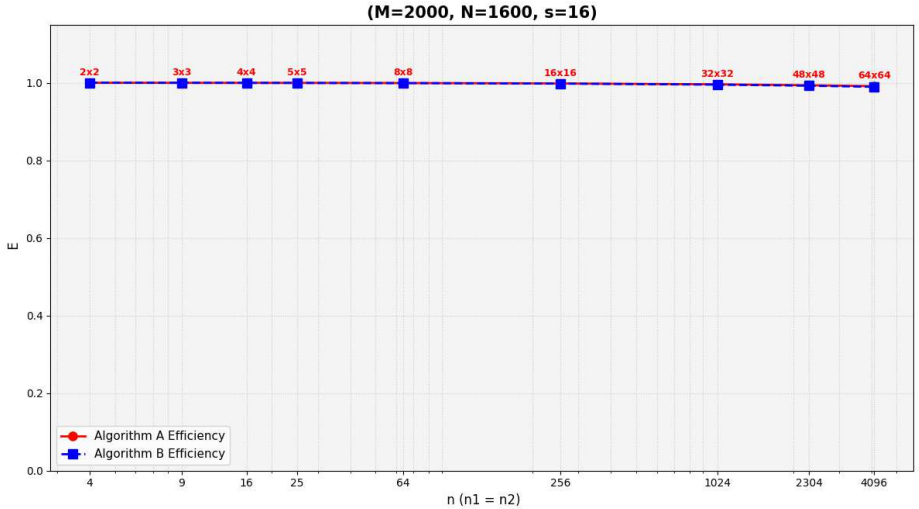


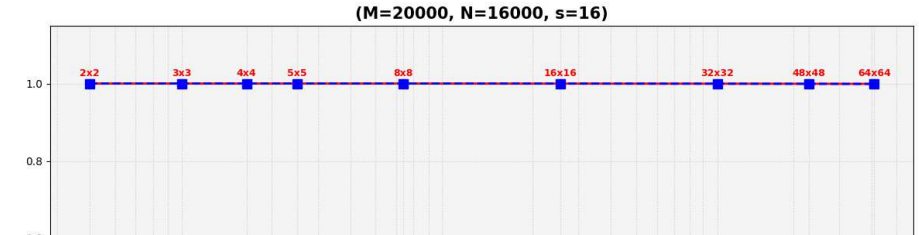


Верх: $M = 200,\ N = 160$ Низ: $M = 2000,\ N = 1600$

Результаты эксперимента: сравнение эффективности







6. Заключение

Наблюдения

В ходе работы были рассмотрены две реализации метода сопряжённых градиентов. Анализ показал, что, **несмотря на различия** в расчётных формулах, эффективность распараллеливания данных версий практически **совпадает**.

Интерпретация

Предложенная методология может быть использована для исследования других модификаций метода на предмет их **параллельной эффективности** и **масштабируемости**.

6. Заключение

Наблюдения

В ходе работы были рассмотрены две реализации метода сопряжённых градиентов. Анализ показал, что, **несмотря на различия** в расчётных формулах, эффективность распараллеливания данных версий практически **совпадает**.

Интерпретация

Предложенная методология может быть использована для исследования других модификаций метода на предмет их **параллельной эффективности** и **масштабируемости**.